Edge AI Lab 1 Report

Pham Minh Long 龍志 Student ID: 314540080

Setup. I ran everything on an NVIDIA GeForce RTX 4090 with `nvcc` 12.1 (host compiler `g++-12`) and Nsight Compute 2022.4. Other people were training models on the same GPU while I measured, so the times move around a bit between runs. In the tables I use the median of 7 runs. The ncu reports were made with the same options as `scripts/profile.sh`. All kernels pass `scripts/check_all.sh`, including memcheck and racecheck.

1 Part 0: Know your GPU

Q1. I took the specs from the TechPowerUp GPU Database and the NVIDIA CUDA GPUs page. The harness prints the same two peaks (82575 GFLOP/s and 1008 GB/s).

Item	Value
GPU	NVIDIA GeForce RTX 4090 (AD102)
Compute capability	8.9
SM count	128 (16,384 CUDA cores)
Boost clock	2.52 GHz
Memory	GDDR6X, 21 Gbps per pin, 384-bit bus
Peak FP32	$2 \times 16384 \times 2.52 = 82{,}575$ GFLOP/s
Peak DRAM bandwidth	$21 \times 384/8 = 1{,}008$ GB/s

Table 1: RTX 4090 specs.

Q2. Ridge point = 82,575/1,008 = 81.9 FLOP/byte. For $M = N = K = 4096$, SGEMM does $2 \cdot 4096^3 = 1.374 \times 10^{11}$ FLOP and moves $4 \cdot 3 \cdot 4096^2 = 201.3$ MB (A and B read once, C written once), so the arithmetic intensity is 682.7 FLOP/byte. This is about 8 times higher than the ridge point, so SGEMM at this size should be compute-bound.

2 Part 1: The naive kernel and the profiler

Q1. Figure 1 is the benchmark output and Table 2 has the medians.

```
(nhan) caotulab@caotulab:~/Desktop/Long/edge-ai/Lab1$ for s in 1024 2048 4096; do echo "== $s"; ./sgemm cublas $s $s $s | tail -1; ./sgemm naive $s $s $s | tail -1; done
== 1024
0   cublas                0.0512    41943.0     245.8   100.0%    50.8%    24.4%  PASS (err 0.0e+00)
1   naive                 3.7509      572.5       3.4     1.4%     0.7%     0.3%  PASS (err 2.3e-05)
== 2048
0   cublas                0.3604    47671.0     139.7   100.0%    57.7%    13.9%  PASS (err 0.0e+00)
1   naive                31.7510      541.1       1.6     1.1%     0.7%     0.2%  PASS (err 0.0e+00)
== 4096
0   cublas                3.0195    45517.8      66.7   100.0%    55.1%     6.6%  PASS (err 0.0e+00)
1   naive               273.3292      502.8       0.7     1.1%     0.6%     0.1%  PASS (err 0.0e+00)
```

Figure 1: Benchmark of `naive` and `cublas`.

Size	cuBLAS ms	cuBLAS GFLOP/s	naive ms	naive GFLOP/s	naive % of peak
1024	0.051	41,764	3.769	570	0.69%
2048	0.360	47,662	33.733	509	0.62%
4096	3.207	42,861	293.635	468	0.57%

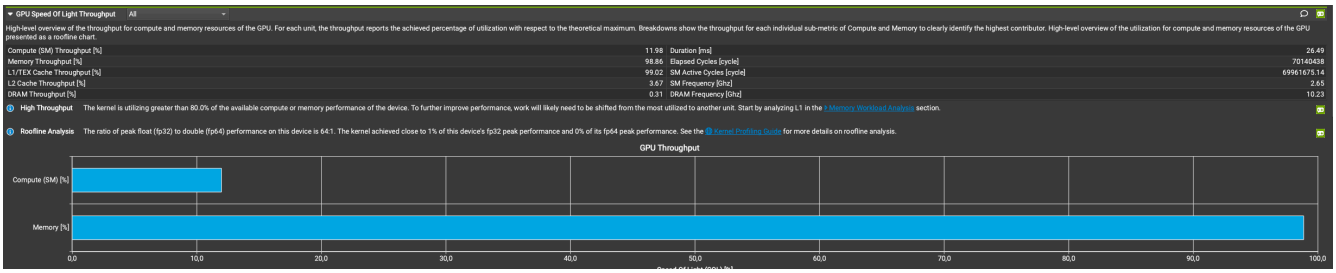
Table 2: Naive kernel against cuBLAS (median of 7 runs).

Q2. I profiled `naive` at 2048 with the ncu command from `scripts/profile.sh` and opened the report in Nsight Compute (Figures 2 to 4).

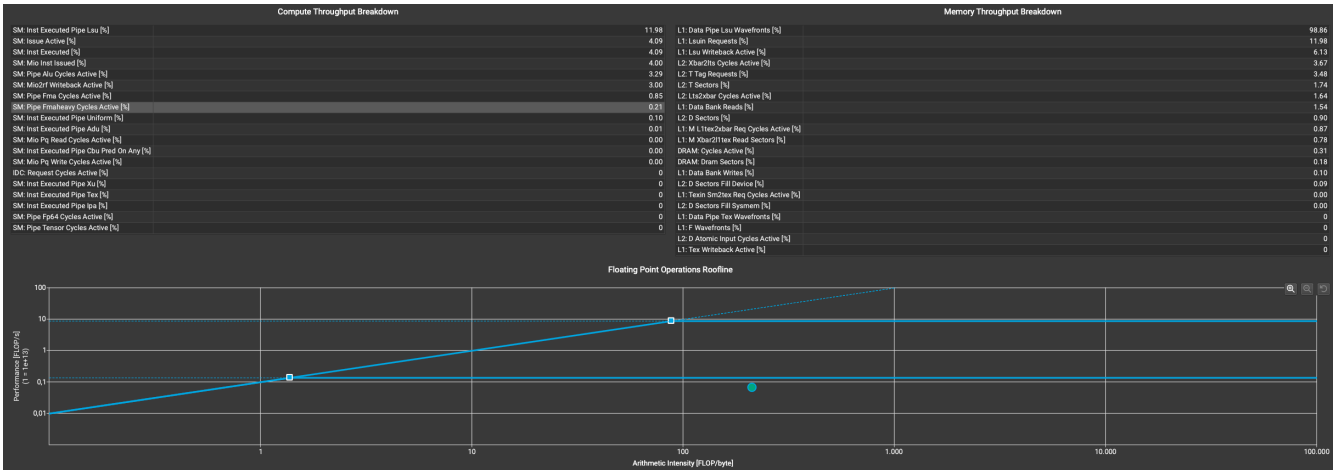
- GPU Speed Of Light: Memory Throughput is 98.86% and Compute (SM) Throughput is 11.98%, so memory is the one close to 100%. Almost all of it is L1/TEX (99.02%). DRAM is only 0.31%.

- Memory Workload Analysis: global loads need 16.5 sectors per request, while a coalesced 4-byte load needs 4. `threadIdx.x` picks the row, so one warp reads A from 32 different rows (32 sectors) and the same element of B (1 sector). On average this is 16.5.
- Warp State Statistics: the main stall is LG Throttle, 180.4 of the 192.1 cycles between two instructions. The FMA pipe is almost idle (0.85%).

So the naive kernel is limited by memory access (L1 and the load queue), not by FP32 math. It reaches 509 GFLOP/s at 2048, which is 0.62% of the peak FP32 GFLOP/s.



(a) Throughput table and GPU Throughput chart.



(b) Compute and memory throughput breakdown and roofline.

Figure 2: GPU Speed Of Light of `naive` at 2048.

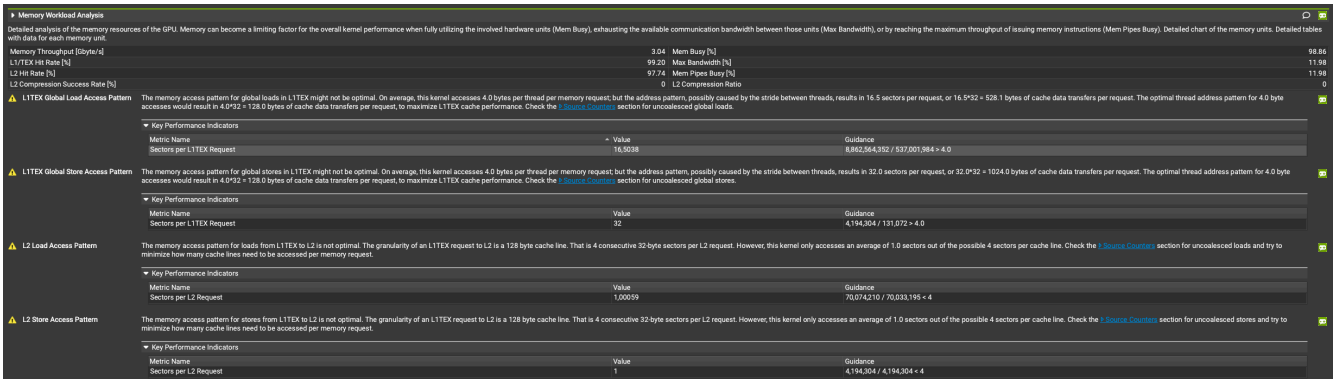


Figure 3: Memory Workload Analysis of `naive` at 2048.

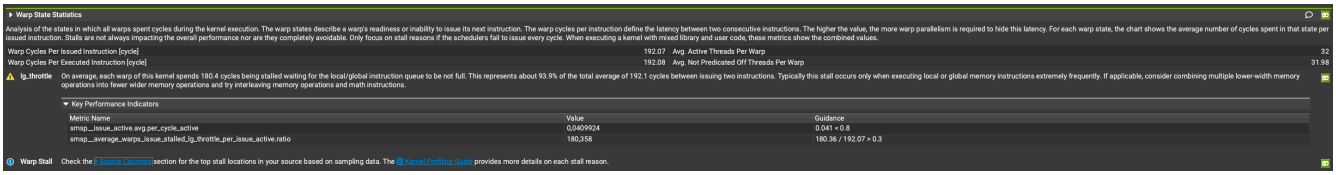


Figure 4: Warp State Statistics of `naive` at 2048.

3 Part 2: Coalescing

In `sgemm_coalesced` I only swapped the index mapping: `col = blockIdx.x * 32 + threadIdx.x` and `row = blockIdx.y * 32 + threadIdx.y`, with the grid `(ceil(N/32), ceil(M/32))`.

Q1. Figure 5 is the benchmark output and Table 3 has the medians.

```
(nhan) caotulab@caotulab:~/Desktop/Long/edge-ai/Lab1$ for s in 1024 2048 4096; do echo "== $s"; ./sgemm naive $s $s $s | tail -1; ./sgemm coalesced $s $s $s | tail -1; done
== 1024
1   naive                4.0141     535.0      3.1     1.3%     0.6%     0.3%  PASS (err 2.3e-05)
2   coalesced            0.4116    5216.8     30.6    12.5%     6.3%     3.0%  PASS (err 2.3e-05)
== 2048
1   naive               32.7557     524.5      1.5     1.1%     0.6%     0.2%  PASS (err 0.0e+00)
2   coalesced            3.5275    4870.2     14.3    10.1%     5.9%     1.4%  PASS (err 0.0e+00)
== 4096
1   naive              273.6190     502.3      0.7     1.0%     0.6%     0.1%  PASS (err 0.0e+00)
2   coalesced           33.4500    4108.8      6.0     9.0%     5.0%     0.6%  PASS (err 0.0e+00)
```

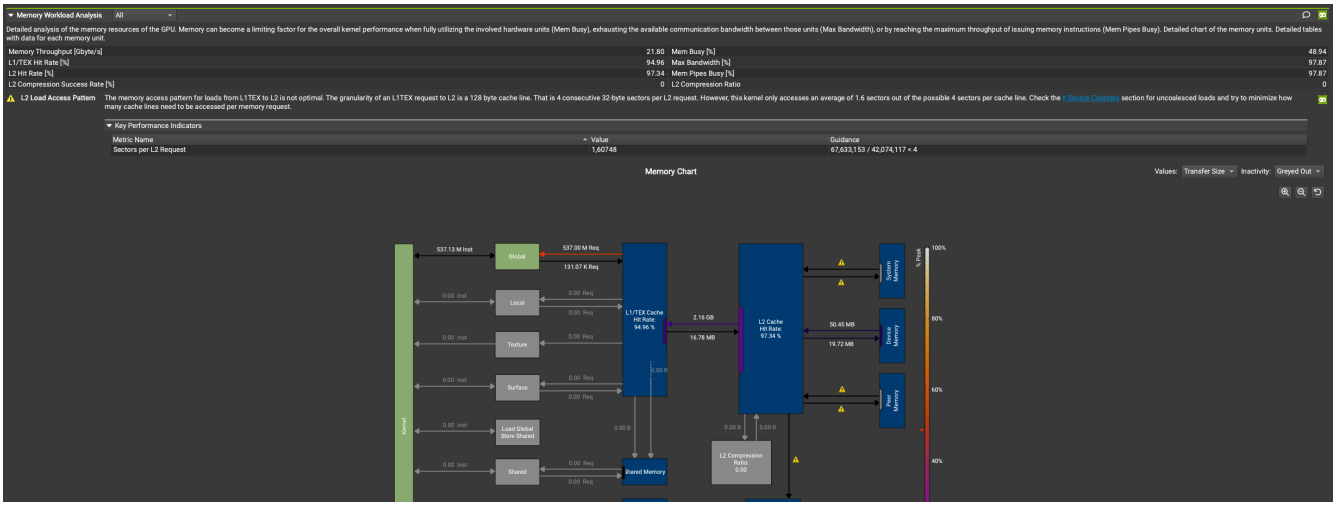
Figure 5: Benchmark of `coalesced`.

Size	naive ms	coalesced ms	coalesced GFLOP/s	Speedup
1024	3.769	0.413	5,204	9.1
2048	33.733	3.827	4,490	8.8
4096	293.635	34.374	3,998	8.5

Table 3: Coalesced kernel against naive (median of 7 runs).

Q2. Both kernels do the same FLOPs and issue the same number of global load requests. The difference is how many sectors each request needs: 16.5 for `naive` and 2.5 for `coalesced` (Figures 3 and 6). Now a warp works on one row, so the 32 threads read the same element of A (1 sector) and 32 consecutive floats of B (4 sectors), which is 2.5 on average. L1 has to move about 6.6 times fewer sectors, so the kernel time in ncu goes from 26.49 ms to 3.22 ms and the LG Throttle stall drops from 180.4 to 17.6 cycles.

Q3. L1 is the busiest level: L1/TEX throughput is 98.99%, L2 is 16.28% and DRAM is 2.20% (Figure 7). The repeated loads of A and B are served from L1 (94.96% hit rate), and most misses hit in L2 (97.34% hit rate). DRAM only reads about 50 MB, close to the 33.6 MB of A and B, so each element comes from DRAM only about once. L1 bandwidth is now the limit, and this is what shared memory tiling improves in Part 3.



(a) Summary table and Memory Chart.

Shared Memory

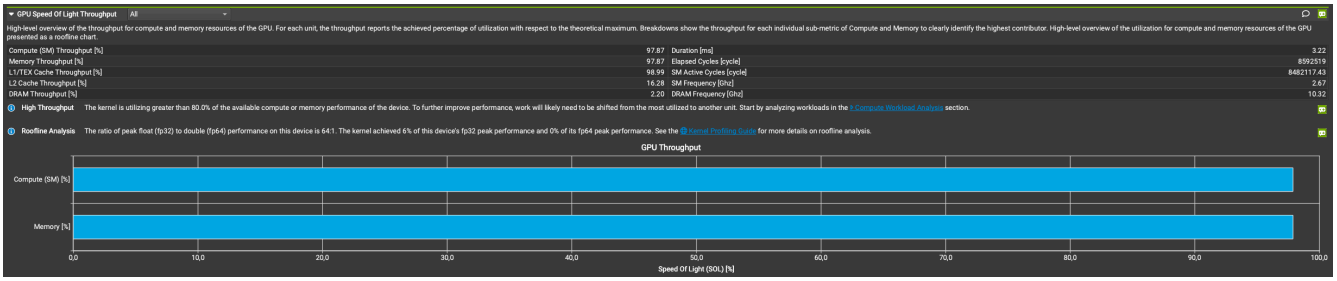
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	0	0			
Shared Load Matrix	0	0	0	0	0
Shared Store	0	0	0	0	0
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	262144	0.02	0
Total	0	0	262144	0.02	0

L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2	Returns to SM	% Peak to SM
Local Load	0	0	0	0	0	0	0	0				
Global Load	537001984	537001984			1342701568	2.50	94.96	42966450176	67633153	6.16	537264128	48.92
Global Load To Shared Store (access)	0	0	537001984	48.90	0	0		0			-	-
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	0	-	-
Surface Load	0	0	0	0	0	0	0	0	0	0	0	0
Texture Load	0	0	0	0	0	0	0	0	0	0		
Global Store	131072	131072	131072	0.01	524288	4	100	16777216	524288	0.05	-	-
Local Store	0	0	0	0	0	0	0	0			-	-
Surface Store	0	0	0	0	0	0	0	0	0	0	-	-
Global Reduction	0	0	0	0	0	0	0	0	0	0	-	-
Surface Reduction	0	0	0	0	0	0	0	0	0	0	-	-
Global Atomic ALU	0											
Global Atomic CAS	0	0	0	0	0	0	0	0	0	0	see above	see above
Surface Atomic ALU	0											
Surface Atomic CAS	0	0	0	0	0	0	0	0	0	0	see above	see above
Loads	537001984	537001984	537001984	48.90	1342701568	2.50	94.96	42966450176	67633153	6.16	537264128	48.92
Stores	131072	131072	131072	0.01	524288	4	100	16777216	524288	0.05	-	-
Atomics & Reductions	0	0	0	0	0	0	0	0	0	0	-	-
Total	537133056	537133056	537133056	48.91	1343225856	2.50	94.96	42983227392	68157441	6.21	537264128	48.92

(b) Shared Memory and L1/TEX Cache tables (Global Load has 2.50 Sectors/Req).

Figure 6: Memory Workload Analysis of `coalesced` at 2048.



(a) Throughput table and GPU Throughput chart.

(b) Roofline.

Figure 7: GPU Speed Of Light of `coalesced` at 2048.

4 Part 3: Tiling with shared memory and registers

3a. In `sgemm_smem<TILE>` each thread loads one element of A and one of B into shared memory per step (0 when it is out of range), then the block syncs, accumulates `TILE` products and syncs again. **3b.** In `sgemm_reg2d` the threads load the A tile in a strided loop and store it transposed (`As[k][m]`), and load the B tile the same way. At the end each thread writes its `TM×TN` results with `C = alpha * acc + beta * C`. I added `128x128x16_8x8`, `64x64x32_4x4` and `128x128x8_4x4` to the three given configurations.

Q1. Figure 8 is the benchmark of all kernels, Figure 10 is the `make` output, and Figures 11 and 12 are the ncu metrics I used in Table 4.

```
(nhan) caotulab@caotulab:~/Desktop/Long/edge-ai/Lab1$ ./sgemm all 4096 4096 4096
GPU: NVIDIA GeForce RTX 4090  (sm_89, 128 SMs, L2 72.0 MB)
Approx. peaks: FP32 82575 GFLOP/s, DRAM 1008 GB/s  (verify against your spec sheet!)
Problem: M=4096 N=4096 K=4096 alpha=1 beta=0

id  kernel                          ms     GFLOP/s      GB/s  %cuBLAS   %peakF  %peakBW  check
0   cublas                       2.7904     49254.2      72.1   100.0%    59.6%     7.2%  PASS (err 0.0e+00)
1   naive                      292.3192       470.2       0.7     1.0%     0.6%     0.1%  PASS (err 0.0e+00)
2   coalesced                   37.2480      3689.8       5.4     7.5%     4.5%     0.5%  PASS (err 0.0e+00)
3   smem16                      27.9419      4918.7       7.2    10.0%     6.0%     0.7%  PASS (err 0.0e+00)
4   smem32                      29.5395      4652.7       6.8     9.4%     5.6%     0.7%  PASS (err 0.0e+00)
5   reg2d_128x128x8_8x8          5.8468     23506.5      34.4    47.7%    28.5%     3.4%  PASS (err 0.0e+00)
6   reg2d_64x64x8_4x4            7.8295     17554.0      25.7    35.6%    21.3%     2.6%  PASS (err 0.0e+00)
7   reg2d_128x64x8_8x4           7.0304     19549.1      28.6    39.7%    23.7%     2.8%  PASS (err 0.0e+00)
8   reg2d_128x128x16_8x8         4.8548     28309.6      41.5    57.5%    34.3%     4.1%  PASS (err 0.0e+00)
9   reg2d_64x64x32_4x4           7.7046     17838.5      26.1    36.2%    21.6%     2.6%  PASS (err 0.0e+00)
10  reg2d_128x128x8_4x4          8.2289     16702.1      24.5    33.9%    20.2%     2.4%  PASS (err 0.0e+00)
11  custom_vec4                  4.2004     32720.1      47.9    66.4%    39.6%     4.8%  PASS (err 0.0e+00)
12  custom_vec4_dbuf             4.0847     33647.0      49.3    68.3%    40.7%     4.9%  PASS (err 0.0e+00)
13  custom_vec4_dbuf_nbc         4.0550     33893.4      49.6    68.8%    41.0%     4.9%  PASS (err 0.0e+00)
```

Figure 8: Benchmark of all kernels at 4096.

Kernel	Time (ms)	GFLOP/s	Regs /thread	Smem /block	Theor. occ.	Achieved occ.	Dominant stall	Bank conflicts load	Bank conflicts store
smem16	25.204	5,453	38	2 KB	100%	99.5%	mio throttle	0	0
smem32	27.859	4,933	38	8 KB	66.7%	66.6%	mio throttle	0	30.0M
reg2d_128x128x8_8x8	4.840	28,394	120	8 KB	33.3%	32.7%	long scoreboard	268.4M	117.4M
reg2d_64x64x8_4x4	7.299	18,830	64	4 KB	66.7%	66.6%	long scoreboard	0	234.9M
reg2d_128x64x8_8x4	7.155	19,208	80	6 KB	50.0%	47.1%	long scoreboard	0	234.9M
reg2d_128x128x16_8x8	4.790	28,690	125	16 KB	33.3%	32.7%	long scoreboard	268.4M	251.7M
reg2d_64x64x32_4x4	7.513	18,293	64	16 KB	66.7%	64.8%	short scoreboard	0.3M	1,054.1M
reg2d_128x128x8_4x4	7.222	19,030	64	8 KB	66.7%	66.6%	long scoreboard	0	121.6M

Table 4: Part 3 results at 4096 (time is the median of 7 runs).

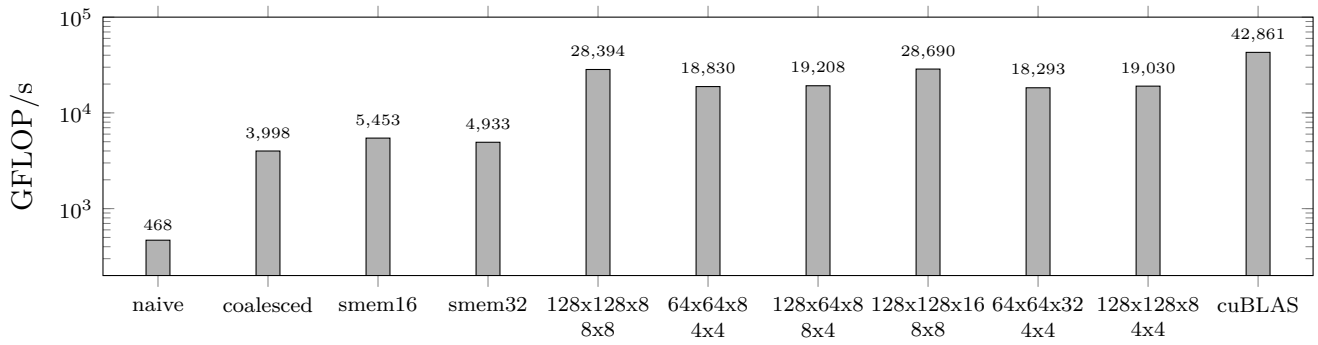


Figure 9: GFLOP/s of each kernel at 4096 (log scale).

Q2. In 3a each FMA needs 2 shared memory loads (`As[ty][k]` and `Bs[k][tx]`). In 3b with TM = TN = 8, each k step loads 8 values of A and 8 of B and does 64 FMAs, so it is 16/64 = 0.25 loads per FMA, 8 times fewer. In `smem16` the shared memory path is the bottleneck: L1 is 96.9% busy, the main stall is MIO Throttle and the FMA pipe is only 11.6% busy. In `reg2d_128x128x8_8x8` the MIO Throttle stall is almost gone and the FMA pipe goes up to 40.6%, so it runs about 5 times faster than `smem16`.

Q3. The fastest one is `reg2d_128x128x16_8x8` (4.790 ms), very close to `reg2d_128x128x8_8x8` (4.840 ms). Both use 8×8 thread tiles and still fit 2 blocks per SM.

```
(nhan) caotulab@caotulab:~/Desktop/Long/edge-ai/Lab1$ make -B NVCC="nvcc -ccbin g++-12"
nvcc -ccbin g++-12 -O3 -std=c++17 -arch=native -use_fast_math -lineinfo -Xptxas -v -o sgemm src/main.cu -lcublas 2> ptxas_info.txt || (cat ptxas_info.txt; false)
Compiling entry function 'void sgemm_vec4_dbuf<(int)128, (int)128, (int)8, (int)8, (int)8, (bool)1>(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 126 registers, 16384 bytes smem, 400 bytes cmem[0]
Compiling entry function 'void sgemm_vec4_dbuf<(int)128, (int)128, (int)8, (int)8, (int)8, (bool)0>(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 128 registers, 16384 bytes smem, 400 bytes cmem[0]
Compiling entry function 'void sgemm_vec4<(int)128, (int)128, (int)8, (int)8, (int)8>(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 115 registers, 8192 bytes smem, 400 bytes cmem[0]
Compiling entry function 'void sgemm_reg2d<(int)128, (int)128, (int)8, (int)4, (int)4>(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 64 registers, 8192 bytes smem, 400 bytes cmem[0]
Compiling entry function 'void sgemm_reg2d<(int)64, (int)64, (int)32, (int)4, (int)4>(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 64 registers, 16384 bytes smem, 400 bytes cmem[0]
Compiling entry function 'void sgemm_reg2d<(int)128, (int)128, (int)16, (int)8, (int)8>(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 125 registers, 16384 bytes smem, 400 bytes cmem[0]
Compiling entry function 'void sgemm_reg2d<(int)128, (int)64, (int)8, (int)8, (int)4>(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 80 registers, 6144 bytes smem, 400 bytes cmem[0]
Compiling entry function 'void sgemm_reg2d<(int)64, (int)64, (int)8, (int)4, (int)4>(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 64 registers, 4096 bytes smem, 400 bytes cmem[0]
Compiling entry function 'void sgemm_reg2d<(int)128, (int)128, (int)8, (int)8, (int)8>(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 120 registers, 8192 bytes smem, 400 bytes cmem[0]
Compiling entry function 'void sgemm_smem<(int)32>(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 38 registers, 8192 bytes smem, 400 bytes cmem[0]
Compiling entry function 'void sgemm_smem<(int)16>(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 38 registers, 2048 bytes smem, 400 bytes cmem[0]
Compiling entry function 'count_bad_guard(const unsigned int *, unsigned long, unsigned long long *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 13 registers, 376 bytes cmem[0]
Compiling entry function 'sgemm_coalesced(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 40 registers, 400 bytes cmem[0]
Compiling entry function 'sgemm_naive(int, int, int, float, const float *, const float *, float, float *)' for 'sm_89'
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
Used 40 registers, 400 bytes cmem[0]
```

Figure 10: Registers and shared memory printed by `make`.

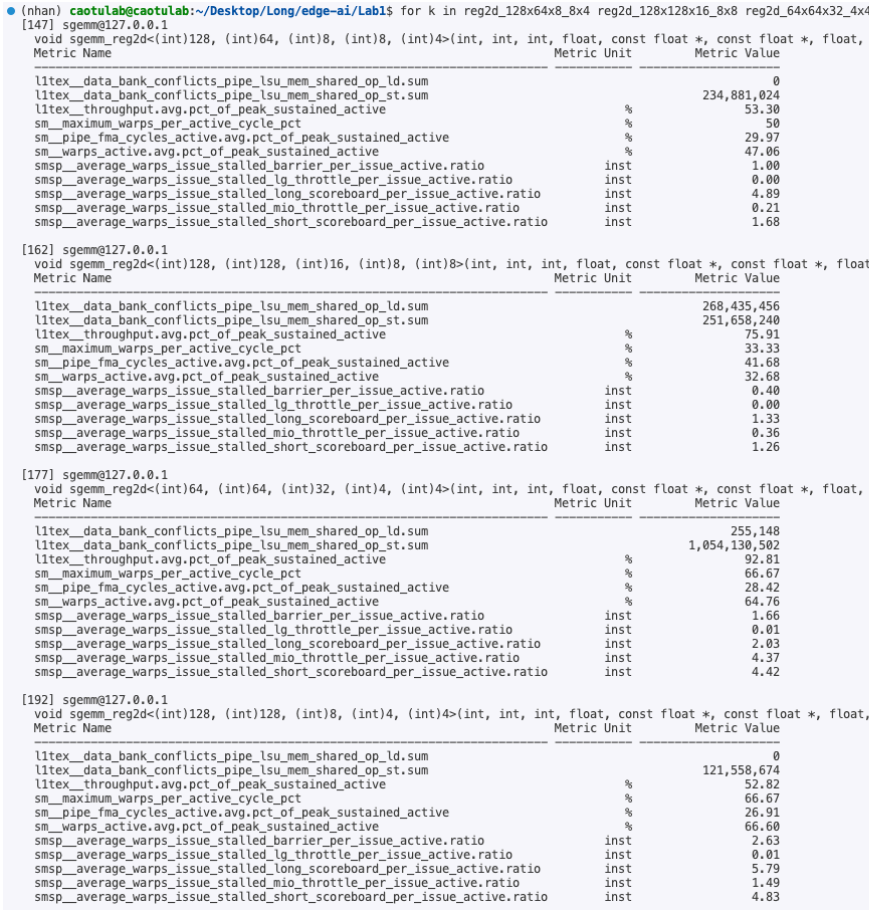
```
(nhan) caotulab@caotulab:~/Desktop/Long/edge-ai/Lab1$ for k in smem16 smem32 reg2d_128x128x8_8x8 reg2d_64x64x8_4x4; do nc
[87] sgemm@127.0.0.1
  void sgemm_smem<(int)16>(int, int, int, float, const float *, const float *, float, float *) (256, 256, 1)x(16, 16, 1),
  Metric Name                                                                  Metric Unit         Metric Value
  ---------------------------------------------------------------------------- ------------ --------------------
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum                                                     0
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum                                                     0
  l1tex__throughput.avg.pct_of_peak_sustained_active                                   %                   96.91
  sm__maximum_warps_per_active_cycle_pct                                               %                     100
  sm__pipe_fma_cycles_active.avg.pct_of_peak_sustained_active                          %                   11.62
  sm__warps_active.avg.pct_of_peak_sustained_active                                    %                   99.50
  smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio                   inst                    5.39
  smsp__average_warps_issue_stalled_lg_throttle_per_issue_active.ratio               inst                    0.02
  smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio           inst                    4.71
  smsp__average_warps_issue_stalled_mio_throttle_per_issue_active.ratio              inst                   18.30
  smsp__average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio          inst                    0.34

[102] sgemm@127.0.0.1
  void sgemm_smem<(int)32>(int, int, int, float, const float *, const float *, float, float *) (128, 128, 1)x(32, 32, 1),
  Metric Name                                                                  Metric Unit         Metric Value
  ---------------------------------------------------------------------------- ------------ --------------------
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum                                                     0
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum                                            30,048,533
  l1tex__throughput.avg.pct_of_peak_sustained_active                                   %                   78.95
  sm__maximum_warps_per_active_cycle_pct                                               %                   66.67
  sm__pipe_fma_cycles_active.avg.pct_of_peak_sustained_active                          %                    8.08
  sm__warps_active.avg.pct_of_peak_sustained_active                                    %                   66.60
  smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio                   inst                    3.51
  smsp__average_warps_issue_stalled_lg_throttle_per_issue_active.ratio               inst                    0.00
  smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio           inst                    3.81
  smsp__average_warps_issue_stalled_mio_throttle_per_issue_active.ratio              inst                   20.30
  smsp__average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio          inst                    0.10

[117] sgemm@127.0.0.1
  void sgemm_reg2d<(int)128, (int)128, (int)8, (int)8, (int)8>(int, int, int, float, const float *, const float *, float,
  Metric Name                                                                  Metric Unit         Metric Value
  ---------------------------------------------------------------------------- ------------ --------------------
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum                                           268,435,456
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum                                           117,440,512
  l1tex__throughput.avg.pct_of_peak_sustained_active                                   %                   67.05
  sm__maximum_warps_per_active_cycle_pct                                               %                   33.33
  sm__pipe_fma_cycles_active.avg.pct_of_peak_sustained_active                          %                   40.63
  sm__warps_active.avg.pct_of_peak_sustained_active                                    %                   32.70
  smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio                   inst                    0.53
  smsp__average_warps_issue_stalled_lg_throttle_per_issue_active.ratio               inst                    0.00
  smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio           inst                    1.20
  smsp__average_warps_issue_stalled_mio_throttle_per_issue_active.ratio              inst                    0.22
  smsp__average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio          inst                    1.15

[132] sgemm@127.0.0.1
  void sgemm_reg2d<(int)64, (int)64, (int)8, (int)4, (int)4>(int, int, int, float, const float *, const float *, float, f
  Metric Name                                                                  Metric Unit         Metric Value
  ---------------------------------------------------------------------------- ------------ --------------------
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum                                                     0
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum                                           234,881,024
  l1tex__throughput.avg.pct_of_peak_sustained_active                                   %                   67.06
  sm__maximum_warps_per_active_cycle_pct                                               %                   66.67
  sm__pipe_fma_cycles_active.avg.pct_of_peak_sustained_active                          %                   29.47
  sm__warps_active.avg.pct_of_peak_sustained_active                                    %                   66.61
  smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio                   inst                    1.59
  smsp__average_warps_issue_stalled_lg_throttle_per_issue_active.ratio               inst                    0.00
  smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio           inst                    6.75
  smsp__average_warps_issue_stalled_mio_throttle_per_issue_active.ratio              inst                    1.04
  smsp__average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio          inst                    2.17
```

Figure 11: ncu metrics for `smem16`, `smem32` and the first two reg2d configurations.



```
(nhan) caotulab@caotulab:~/Desktop/Long/edge-ai/Lab1$ for k in reg2d_128x64x8_8x4 reg2d_128x128x16_8x8 reg2d_64x64x32_4x4
[147] sgemm@127.0.0.1
  void sgemm_reg2d<(int)128, (int)64, (int)8, (int)8, (int)4>(int, int, int, float, const float *, const float *, float,
  Metric Name                                                                  Metric Unit         Metric Value
  ---------------------------------------------------------------------------- ----------- --------------------
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum                                                       0
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum                                             234,881,024
  l1tex__throughput.avg.pct_of_peak_sustained_active                                   %                   53.30
  sm__maximum_warps_per_active_cycle_pct                                               %                      50
  sm__pipe_fma_cycles_active.avg.pct_of_peak_sustained_active                          %                   29.97
  sm__warps_active.avg.pct_of_peak_sustained_active                                    %                   47.06
  smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio                   inst                    1.00
  smsp__average_warps_issue_stalled_lg_throttle_per_issue_active.ratio               inst                    0.00
  smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio           inst                    4.89
  smsp__average_warps_issue_stalled_mio_throttle_per_issue_active.ratio              inst                    0.21
  smsp__average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio          inst                    1.68

[162] sgemm@127.0.0.1
  void sgemm_reg2d<(int)128, (int)128, (int)16, (int)8, (int)8>(int, int, int, float, const float *, const float *, float
  Metric Name                                                                  Metric Unit         Metric Value
  ---------------------------------------------------------------------------- ----------- --------------------
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum                                             268,435,456
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum                                             251,658,240
  l1tex__throughput.avg.pct_of_peak_sustained_active                                   %                   75.91
  sm__maximum_warps_per_active_cycle_pct                                               %                   33.33
  sm__pipe_fma_cycles_active.avg.pct_of_peak_sustained_active                          %                   41.68
  sm__warps_active.avg.pct_of_peak_sustained_active                                    %                   32.68
  smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio                   inst                    0.40
  smsp__average_warps_issue_stalled_lg_throttle_per_issue_active.ratio               inst                    0.00
  smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio           inst                    1.33
  smsp__average_warps_issue_stalled_mio_throttle_per_issue_active.ratio              inst                    0.36
  smsp__average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio          inst                    1.26

[177] sgemm@127.0.0.1
  void sgemm_reg2d<(int)64, (int)64, (int)32, (int)4, (int)4>(int, int, int, float, const float *, const float *, float,
  Metric Name                                                                  Metric Unit         Metric Value
  ---------------------------------------------------------------------------- ----------- --------------------
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum                                             255,148
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum                                           1,054,130,502
  l1tex__throughput.avg.pct_of_peak_sustained_active                                   %                   92.81
  sm__maximum_warps_per_active_cycle_pct                                               %                   66.67
  sm__pipe_fma_cycles_active.avg.pct_of_peak_sustained_active                          %                   28.42
  sm__warps_active.avg.pct_of_peak_sustained_active                                    %                   64.76
  smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio                   inst                    1.66
  smsp__average_warps_issue_stalled_lg_throttle_per_issue_active.ratio               inst                    0.01
  smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio           inst                    2.03
  smsp__average_warps_issue_stalled_mio_throttle_per_issue_active.ratio              inst                    4.37
  smsp__average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio          inst                    4.42

[192] sgemm@127.0.0.1
  void sgemm_reg2d<(int)128, (int)128, (int)8, (int)4, (int)4>(int, int, int, float, const float *, const float *, float,
  Metric Name                                                                  Metric Unit         Metric Value
  ---------------------------------------------------------------------------- ----------- --------------------
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum                                                       0
  l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum                                             121,558,674
  l1tex__throughput.avg.pct_of_peak_sustained_active                                   %                   52.82
  sm__maximum_warps_per_active_cycle_pct                                               %                   66.67
  sm__pipe_fma_cycles_active.avg.pct_of_peak_sustained_active                          %                   26.91
  sm__warps_active.avg.pct_of_peak_sustained_active                                    %                   66.60
  smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio                   inst                    2.63
  smsp__average_warps_issue_stalled_lg_throttle_per_issue_active.ratio               inst                    0.01
  smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio           inst                    5.79
  smsp__average_warps_issue_stalled_mio_throttle_per_issue_active.ratio              inst                    1.49
  smsp__average_warps_issue_stalled_short_scoreboard_per_issue_active.ratio          inst                    4.83
```

Figure 12: ncu metrics for the other four reg2d configurations.

A clearly slower one is `reg2d_64x64x32_4x4` (7.513 ms). Its occupancy is higher (66.7% against 33.3%), but it is still 1.57 times slower. With 4×4 tiles it needs 0.5 shared memory loads per FMA. Also, with BK = 32 the transposed store of A puts all 32 threads of a warp on the same bank, and ncu reports about 1,054M store bank conflicts (252M for the fastest one). Because of this its main stalls are Short Scoreboard and MIO Throttle, L1 is 92.8% busy and the FMA pipe is only 28.4% busy (41.7% for the fastest one). `reg2d_128x128x8_4x4` is slow for another reason: with 1024 threads per block only one block fits on an SM, and the warps spend a lot of time waiting at the barriers and for global loads.

5 Part 4: Your own optimizations

My chain is `reg2d_128x128x8_8x8`, `custom_vec4`, `custom_vec4_dbuf` and `custom_vec4_dbuf_nbc`. The custom kernels keep BM = BN = 128, BK = 8 and TM = TN = 8, so each step adds one change. Figure 13 is the output of `scripts/grade.sh` and Figure 15 has the ncu metrics.

```
(nhan) caotulab@caotulab:~/Desktop/Long/edge-ai/Lab1$ PATH=~/tools/cuda_sanitizer_api-linux-x86_64-12.8.93-archive/compute-sanitizer:$PATH scripts/grade.sh | tee ~/grade_output.txt
== correctness: 15 shapes x 2 betas
   128 128 128 done
   1024 1024 1024 done
   4096 4096 4096 done
   1000 1000 1000 done
   1023 257 511 done
   1 1 1 done
   7 13 3 done
   4096 1 4096 done
   1 4096 4096 done
   8192 1 8192 done
   16 4096 4096 done
   4096 4096 64 done
   64 64 65536 done
   1000 1 1001 done
   4097 1 4099 done
== sanitizer: 12 kernels x 8 runs

== Part 2
   coalesced                  PASS
== Part 3
   smem16                     PASS
   smem32                     PASS
   reg2d_128x128x8_8x8        PASS
   reg2d_64x64x8_4x4          PASS
   reg2d_128x64x8_8x4         PASS
   reg2d_128x128x16_8x8       PASS
   reg2d_64x64x32_4x4         PASS
   reg2d_128x128x8_4x4        PASS
   reg2d configurations       6 correct of 6 registered (need >= 6): PASS
== Part 4 (chain at 4096 4096 4096, best of 3 x median of 20)
   0 reg2d_128x128x8_8x8         4.602 ms  starting point
   1 custom_vec4                 3.282 ms  1.402x vs reg2d_128x128x8_8x8: successful, 5 pts
   2 custom_vec4_dbuf            3.500 ms  0.938x vs custom_vec4: failed (needs an explanation in the report), 2.5 pts
   3 custom_vec4_dbuf_nbc        3.301 ms  1.060x vs custom_vec4_dbuf: successful, 5 pts
   Part 4 points: 10 / 10
```

Figure 13: Output of `scripts/grade.sh`.

Step	Change	Time (ms)	Speedup	Result
reg2d_128x128x8_8x8	baseline	4.602	none	starting point
custom_vec4	float4 loads and stores	3.282	1.402	successful
custom_vec4_dbuf	double buffering	3.500	0.938	failed
custom_vec4_dbuf_nbc	no bank conflicts on `Bs`	3.301	1.060	successful

Table 5: Part 4 chain at 4096 from `scripts/grade.sh`.

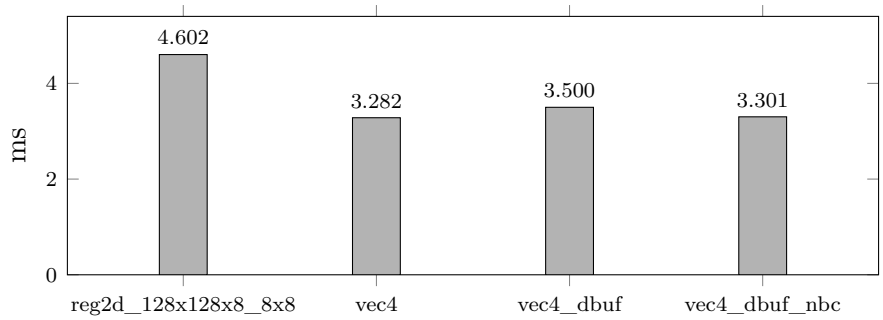


Figure 14: Time of each step at 4096.

Step 1: float4 loads (`custom_vec4`), successful. I changed the global loads, the shared memory reads and the writes to C to `float4`. The time went from 4.602 ms to 3.282 ms (1.402 times faster). It worked because the kernel needs fewer instructions for the same data. Global load requests went from 34.1M to 8.5M and the FMA pipe went from 40.6% to 56.4% busy. It uses 115 registers, so 2 blocks still fit on one SM.

Step 2: double buffering (`custom_vec4_dbuf`), failed. I added a second shared memory buffer. The loads for the next tile are issued into registers before the computation of the current tile, and written to the other buffer after it, so only one `__syncthreads()` per tile is needed. In ncu the kernel waits much less for global memory: the Long Scoreboard stall dropped from 1.01 to 0.06 and the FMA pipe went from 56.4% to 63.3%. But in `grade.sh` the time went from 3.282 ms to 3.500 ms (0.938 times), so this step failed. I think the main reason is that `custom_vec4` already runs 2 blocks (16 warps) per SM. When one block waits for global loads or at the barrier, the other block can still compute, so most of the load latency was already hidden and there was not much left to gain. The prefetch also costs more registers (115 to 128) and twice the shared memory (8 KB to 16 KB). The GPU was shared with other jobs, and

```
template <int BM, int BN, int BK, int TM, int TN>
__device__ __forceinline__ void load_global_vec4(
    const float* __restrict__ A, const float* __restrict__ B, int N, int K,
    int k0, TileLoad<BM, BN, BK, TM, TN>& regs) {
  constexpr int NT = TileLoad<BM, BN, BK, TM, TN>::kThreads;
  const int tid = threadIdx.x;
#pragma unroll
  for (int r = 0; r < TileLoad<BM, BN, BK, TM, TN>::kA; ++r) {
    const int i = tid + r * NT;
    const int m = i / (BK / 4);
    const int k = (i % (BK / 4)) * 4;
    regs.a[r] = *reinterpret_cast<const float4*>(
        &A[(size_t)(blockIdx.y * BM + m) * K + k0 + k]);
  }
#pragma unroll
  for (int r = 0; r < TileLoad<BM, BN, BK, TM, TN>::kB; ++r) {
    const int i = tid + r * NT;
    const int k = i / (BN / 4);
    const int n = (i % (BN / 4)) * 4;
    regs.b[r] = *reinterpret_cast<const float4*>(
        &B[(size_t)(k0 + k) * N + blockIdx.x * BN + n]);
  }
}
```

Code 1: Global loads of the A and B tiles with `float4` (`custom.cuh`).

```
  float ra[TM], rb[TN];
#pragma unroll
  for (int k = 0; k < BK; ++k) {
#pragma unroll
    for (int i = 0; i < TM; i += 4) {
      const float4 a = *reinterpret_cast<const float4*>(&As[k][tr * TM + i]);
      ra[i + 0] = a.x;
      ra[i + 1] = a.y;
      ra[i + 2] = a.z;
      ra[i + 3] = a.w;
    }
#pragma unroll
    for (int j = 0; j < TN; j += 4) {
      const float4 b = *reinterpret_cast<const float4*>(
          &Bs[k][thread_col<BN, TN, STRIDED_N>(tc, j)]);
      rb[j + 0] = b.x;
      rb[j + 1] = b.y;
      rb[j + 2] = b.z;
      rb[j + 3] = b.w;
    }
#pragma unroll
    for (int i = 0; i < TM; ++i)
#pragma unroll
      for (int j = 0; j < TN; ++j) acc[i][j] += ra[i] * rb[j];
  }
```

Code 2: Inner loop: `float4` reads from shared memory, then the 8×8 outer product.

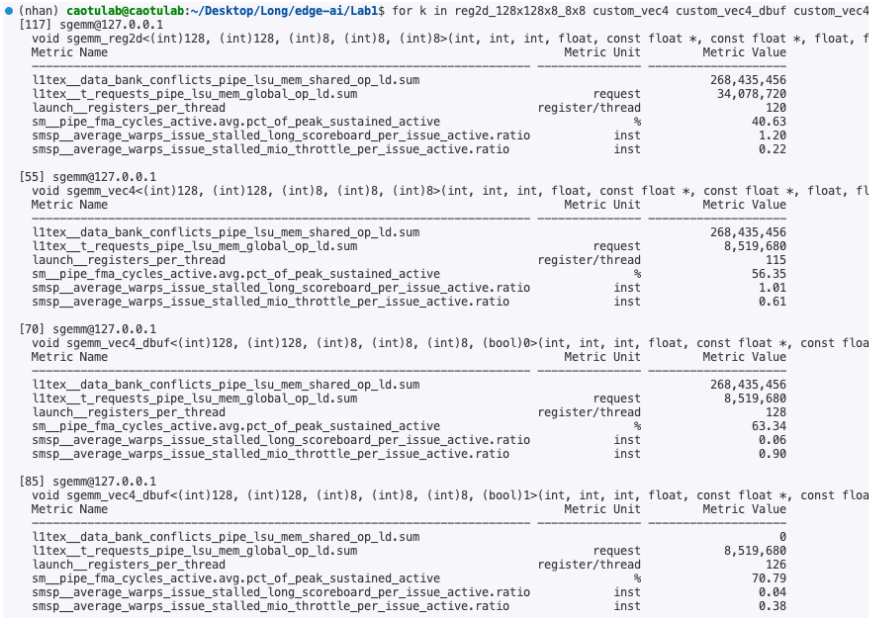


Figure 15: ncu metrics for the four kernels of the chain.

in my earlier runs this step went from 0.94 to 1.17 times, so its real effect is small and easily covered by the noise.

```
__shared__ __align__(16) float As[2][BK][BM];
__shared__ __align__(16) float Bs[2][BK][BN];
const int tid = threadIdx.x;
const int tr = tid / (BN / TN);
const int tc = tid % (BN / TN);
float acc[TM][TN] = {};
TileLoad<BM, BN, BK, TM, TN> next;
load_global_vec4<BM, BN, BK, TM, TN>(A, B, N, K, 0, next);
store_shared_vec4<BM, BN, BK, TM, TN>(next, As[0], Bs[0]);
__syncthreads();
int stage = 0;
for (int k0 = 0; k0 < K; k0 += BK) {
  const bool has_next = k0 + BK < K;
  if (has_next) {
    load_global_vec4<BM, BN, BK, TM, TN>(A, B, N, K, k0 + BK, next);
  }
  outer_product_vec4<BM, BN, BK, TM, TN, STRIDED_N>(As[stage], Bs[stage], tr,
                                                    tc, acc);
  if (has_next) {
    store_shared_vec4<BM, BN, BK, TM, TN>(next, As[stage ^ 1], Bs[stage ^ 1]);
  }
  __syncthreads();
  stage ^= 1;
}
store_tile_vec4<BM, BN, TM, TN, STRIDED_N>(C, N, alpha, beta, tr, tc, acc);
```

Code 3: Main loop of `sgemm_vec4_dbuf` with two shared memory stages.

Step 3: no bank conflicts on `Bs` (`custom_vec4_dbuf_nbc`), successful. With 8 contiguous columns per thread, the reads of `Bs` had 268M bank conflicts. I changed the mapping so each thread takes columns `tc*4` to `tc*4+3` and `64+tc*4` to `64+tc*4+3`, so a warp reads contiguous memory. The bank conflicts went to 0, the MIO Throttle stall dropped from 0.90 to 0.38 and the FMA pipe went up from 63.3% to 70.8%. The time went from 3.500 ms to 3.301 ms (1.060 times faster), so this step passed the 1.05 line. It worked because each shared memory read now needs fewer wavefronts, so the warps can issue FMAs more often.

The gain is still small, because after double buffering most shared memory reads were already overlapped with the FMAs.

```
template <int BN, int TN, bool STRIDED_N>
__device__ __forceinline__ int thread_col(int tc, int j) {
  if (STRIDED_N) {
    return (j / 4) * (BN / TN) * 4 + tc * 4 + j % 4;
  }
  return tc * TN + j;
}
```

Code 4: Column mapping. `STRIDED_N = true` is used by `custom_vec4_dbuf_nbc`.